Agentic Coding with VS Code

Center for Integrated Cognition

Bob Wray

APR 2026

Goals for Today

- Various ways to exploit modern AI tools for computational research
  - Large language models (LLMs), foundation models, “AI”, …

intro • Direct interaction with AI Agent

intro • Tools that encapsulate various research functions

deep • Help you develop traditional computer code (exploration, analysis, presentation)

intro (if/as time) • Exploit LLMs within your code (“agentic code”)

(c) 2026 Robert Wray

Direct Interaction with AI Agent

- Standard “chat” interface we are all familiar with
  - Gemini, ChatGPT, Perplexity/Comet, etc.

- Cost: Free-$20/month
  (typically, only a few high-quality responses/day)

- Uses for Research:
  - Unlike first versions of ChatGPT, currently queries generally embed some web search with “RAG”-like response and even reasoning
  - Great for general awareness/covering the waterfront

(c) 2026 Robert Wray

Research-specific Tools

- Many people are developing tools around LLMs that are intended to encapsulate various research functions
- Example: Google NotebookLM
  - Answer questions about a collected corpus
  - Cost: ~Free (limited to 50 docs)
- Deep Research: OpenAI
  - Uses web search and reasoning to produce research reports
  - Limited but useful free functionality (~5 requests/month with free account)
- UNC-specific resources: https://guides.lib.unc.edu/GenAI/tools
- Some commercial ventures (not endorsement/further review)
  - Edison Scientific: https://edisonscientific.com/
  - Future House: https://www.futurehouse.org/

(c) 2026 Robert Wray

Using AI to Support Code Development

- Coding models: Models trained on a larger mix of code (and other text) with the goal to support coding tasks explicitly
  - OpenAI Codex, Google Gemma
  - Claude and Gemini: both considered general-purpose models but emphasize coding
- Coding environments and tools
  - Platforms that allow you to exploit coding models
  - Many coding environments allow choice of models
  - Examples: **VS Code**, Claude Code, Cursor, etc.
  - Cost: $20-200/month. (Free versions are not useful for sustained work.)
- Command-line Interface (CLI) environments:
  - Popularized by Claude Code
  - More for “vibe coding” (assume that you will not review/inspect the code yourself)
  - The more bespoke your coding task, the more likely it is you won’t succeed purely vibe-coding

(c) 2026 Robert Wray

How a Coding Model Processes a Conversational Turn

CONTEXT WINDOW (e.g. 200 k tokens)

System Instructions
- Custom agent rules
- Coding guidelines
- Repo conventions
- Security policies
- Output format rules

Skill Files
- SKILL.md headers
- Workflow templates
- Domain knowledge
- Tool call patterns
- Best practices

Prior Interactions
- Previous user turns
- Previous AI replies
- Tool call results
- Error outputs
- Multi-turn context

File Attachments
- Code files
- Data files
- Images
- Notebooks
- Docs / PDFs

Workspace Context
- Open editor files
- Directory structure
- Git branch info
- Active terminal
- Error diagnostics

Tool Definitions
- File read / write
- Web search
- Terminal exec
- Semantic search
- Memory tools

Retrieval Results
- Semantic code search
- grep matches
- Memory reads
- Web fetch output
- DB query results

Current Prompt ★
- **User message**
- **@ mentions**
- **Attached images**
- **Editor selection**

Tokenized Context

Instructions & Skills	Prior Interactions	Workspace & Tools	Attachments & Retrieval	Current Prompt	Available for Output

Token budget consumed per turn (proportional illustration)

Language Model

(Transformer / Attention)

Tokenizer
Converts text → token IDs

Attention Layers
Self-attention over full context

Feed-Forward Layers
Per-token transformations

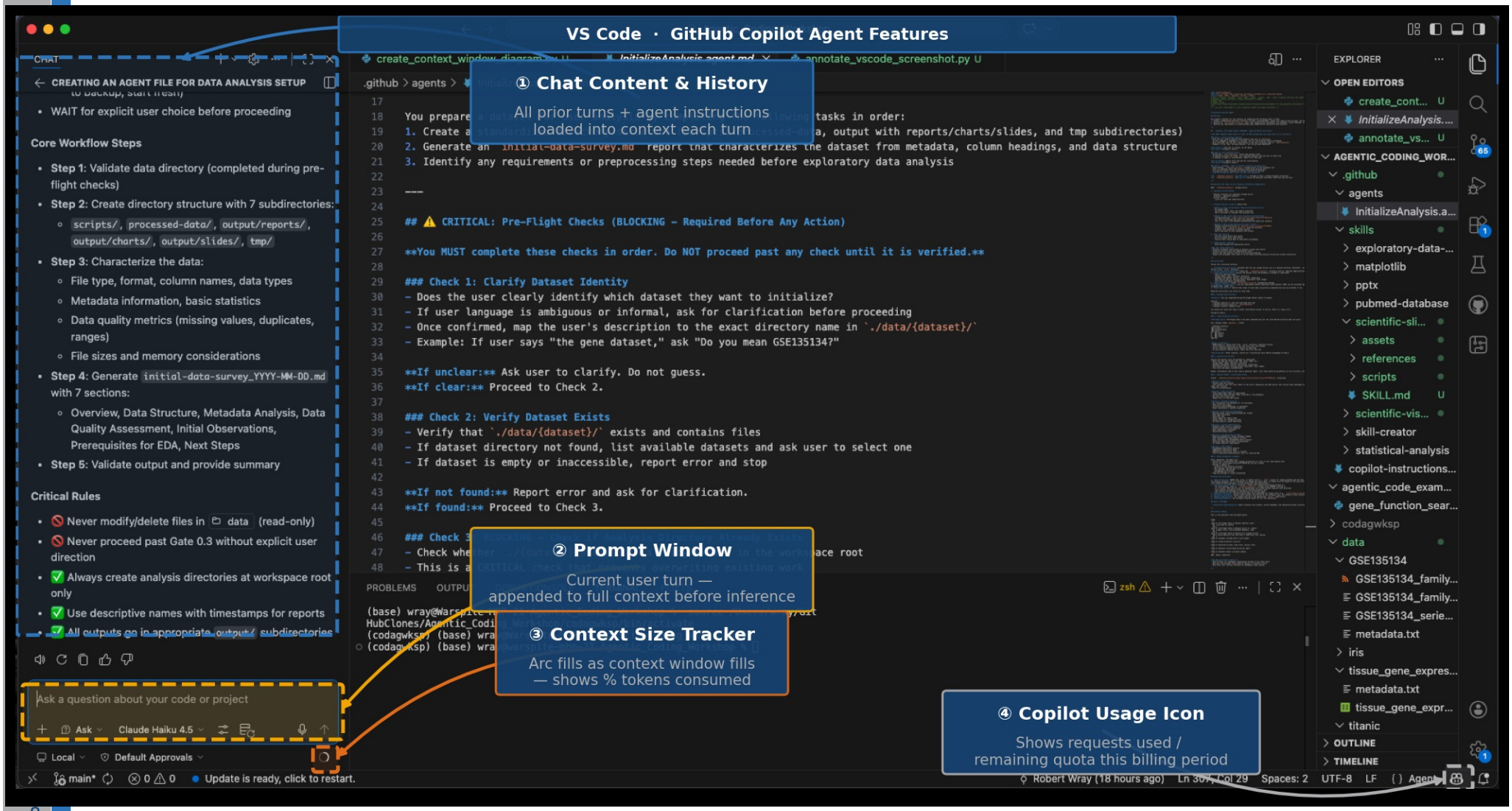
Output Head
Predicts next token (logits)

Sampler / Tool Call
Decodes response or issues a tool call → re-enters context

Generated Response / Tool Call

Tool results re-enter context on next turn

★ Only the Current Prompt changes turn-to-turn; all other context components persist and consume token budget.



(c) 2026 Robert Wray

GitHub Copilot

Coding assistant built into VS Code

- Code completion
- Inline coding support
- Agent modes
  - Plan (read only): Map out a large project and identify specific tasks
  - Ask (read only): Ask questions about a code base, design, problem, etc.
  - Agent (read/write): Write, run code, commits, etc.
  - Custom Agents: Custom workflows you define for agents to follow

Customizable in many ways….

- Choice of model
- Development goals and philosophy (copilot-instructions)
- Skills
- Permissions and access

(c) 2026 Robert Wray

GitHub Copilot

Coding assistant built into VS Code

- Code completion
- Inline coding support
- Agent modes
  - Plan (read only): Map out a large project and identify specific tasks
  - Ask (read only): Ask questions about a code base, design, problem, etc.
  - Agent (read/write): Write, run code, commits, etc.
  - **Custom Agents**: Custom workflows you define for agents to follow

Customizable in many ways….

- Choice of model
- Development goals and philosophy (copilot-instructions)
- **Skills**
- Permissions and access

(c) 2026 Robert Wray

What Model Should I Use (VS Code)

General rule: Use the smallest model that can do the task. Why?

- “Standard” VS Code account: ~300 “premium requests”/month
  - VS Code includes code completions and inline suggestions (unlimited)
  - “Premium request” is somewhat the same as a chat turn but complex turns can turn into multiple requests
  - Examples: Haiku 4.5 (x.33), Sonnet (x1), Opus 4.6 (x3)
- Over time, you will develop a “feel” for which models can do what
  - Bob generally uses Sonnet 4.6 as default
  - Haiku for basic file operations, git operations, and implementing and running tests
  - Opus mostly for systems-level planning (multi-phase projects)
    - Be especially careful using frontier models for open-ended tasks; they will act autonomously and perform many requests in one turn
  - If the task is not unusual and not demanding, a smaller model can likely perform it.

(c) 2026 Robert Wray

Customizing Workflows

- Copilot Instructions
  - Overall instructions for a project
  - What rules should the agent always follow
  - What is important to you (rapid prototyping, maintainability, error handling, etc.)
- Skills
  - Consistent approach to multi-step tasks
  - Include scripts, templates, etc. for encourage consistent use
- Custom Agents
  - Multistep tasks that involve decision making (where you want to inform the decision-making process)

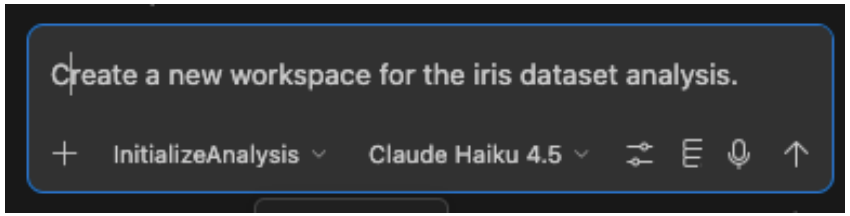
- All found in `./.github/`
- Represented in natural language (easy to understand and edit)
- Many online repositories of skills
  - See https://skills.sh
  - Not covered today but repo includes a meta-skill for making new skills (skill-creator)

(c) 2026 Robert Wray

Data Analysis Agentic_Coding_Workshop — Repository Structure

Exercise #1: Custom Agent

1. Read thru what the InitializeAnalysis does
2. Enable the agent in your agent list…
3. Choose the InitializeAnalysis agent and model (A .33x model should be fine)
4. Ask the agent to create a workspace for the iris dataset. Note:
   - Permissions and allowed actions
   - “Keep”
   - Changes to file system



5. Review the report on the iris dataset
   - Use “Open Preview” (CMD-SHIFT-V) to view the markdown rendering of the report
6. Create a new chat and repeat for titantic or tissue datasets

(c) 2026 Robert Wray

Exercise #2: Skills (EDA)

> Follow instructions in SKILL.md.
> perform EDA on the dataset, focusing especially on the basic properties of the data and identifying any significant issues using the data. Do not duplicate the analysis in the initial report (attached). Generate a summary report (in output) that includes your findings and recommended next steps.

1. Return to the chat where you created the iris dataset
2. Scan the `exploratory-data-analysis` skill
   (How will you change it for *your* process?)
3. Change the agent to "Plan"
4. Tell it what you want to know about the dataset / your goals for EDA
   - Take a few conversation turns if you wish
   - Make sure it is using the EDA skill. If not invoke it directly using "/"
5. When finished setting goals, switch to Agent mode
6. Ask the agent to complete the EDA as planned and save the results as a report
7. Read the report – how did the agent do?
8. Note: How much of the context have you consumed with these two steps?
9. BONUS: Ask the agent to create a PPT summarizing the report (use /PPTX)

(c) 2026 Robert Wray

Context Management

- General goal: Help the model manage its ever-growing context
- More “frontier” models have not only more capability but generally have larger context windows
  - But using frontier models for basic tasks will exhaust token budget quickly
- Strategies:
  - Create intermediate work products to use as “references” for new chats
  - Switch to new chats when the task changes (plan, create, debug, finalize presentation, etc.)
  - Be specific when referencing files and directories (use attachments)
  - Explicitly call out skills when you want them used
  - ...
- Note: Several recent, frontier models are now automating aspects of strategy #1

(c) 2026 Robert Wray

Exercise #3: Skills (Matplotlib)

1. Scan the `matplotlib` skill. Note the styleguide. (How will you change it for *your* process?)
2. Pick out a plot from the EDA for the agent to build.
   - Tell the agent to build it (invoke the skill)
   - Provide guidance for any constraints (Colors, size, aspect ratio, etc.)
3. Note:
   1. Where is the output file located?
   2. Where is the source script located?
4. Change something manually in the source script (e.g., title) and rerun it
   - Generally, the agent will design code to run from the root directory:
   - `python iris_analysis/scripts/plot_petal_scatter.py`
   - Results in `iris_analysis/outputs/charts/petal_scatter.png`

> Follow instructions in SKILL.md.
> Let's createthe pairplot recommended in the report. I would to use a colorblind friendly palette using the same colors for all spiecies across all charts.

Why is it doing this? How can you make it do something different?

(c) 2026 Robert Wray

Exercise #4: Skills (Scientific Vis.)

1. Scan the `scientific-visualization` skill.
   (How will you change it for *your* process? What journals need to be added?)
2. Choose one of the plots from exercise #3.
3. Tell the agent to re-create the chosen plot for a specific journal (Science, Nature, etc.)
   - Provide guidance for any constraints (Colors, size, aspect ratio, etc.)
   - Ask it to ask you clarification questions before plotting. What does it ask?
4. Repeat step #3 for a different journal
5. Note: How do the final products differ?

> Follow instructions in SKILL.md.
> Let's revisit the scatterplot and prepare it for publication in Science. Create a new version of the plotfile that conforms to Science styles and conventions. Ask questions to clarify any unknowns before creating the plot.

(c) 2026 Robert Wray

Additional Topics for Further Research

- Custom instructions (somewhat deprecated?)
- MCP: Model context protocol
  - Roughly, “APIs” for language models
  - Mostly relevant for agentic applications
- Subagents/Task Delegation
  - Agents can “assign” tasks to other agents (and work in parallel)
  - More model-autonomous approach to language models
- Hooks
  - Run scripts based on specific events in the agent coding loop
  - Example: Do not try to run a test until code lints
- Skill Composition
  - Methods to chain skills together
- …

(c) 2026 Robert Wray

Using LLMs within Code

- We've been using LLMs to write standard code
- Can also embed LLM function calls within code
  - "Agentic" just means that there is some model-mediated conditionality in the code
- Many tools being created to support agentic code development
  - Langchain/Langraph: Dynamic workflows via LLMs
  - Contextual: Visual programming
  - DSPy: Encapsulate prompting ("programming not prompting")
- Demo
  - Implemented with DSPy
  - Creates search terms for Wikipedia
  - Extracts gene.
  - Interprets results and provides a rationale for the chosen gene.

Code is in the repo but requires an API key to run it yourself…

(c) 2026 Robert Wray